Security-Preserving Optimizations

CS-428 project report

Bennani Maria Carraux Guillaume Perazzolo Sean

2026-05-31

Abstract

Standard compiler optimizations can break important security properties when only focusing on functional correctness. This project formalizes, in Rocq, a framework to reason about secure optimizations, closing the gap between correctness and security. The framework targets two main guarantees: an optimization must not remove deliberate erasure assignments that clear sensitive memory, and it must not introduce secret-dependent observable behavior, so that traces remain independent of secret inputs. Using this framework, the project proves that a dead-store-elimination pass is both semantically sound and security-preserving. The framework is designed to be extensible, making it straightforward to incorporate and verify additional optimization passes in the same setting.

Keywords: Big-Step Semantics, Interpreters, Rule Induction, Soundness, Proof Automation

Introduction

Standard optimization tools assume functional correctness is the only required property. However, this approach can introduce security vulnerabilities. For instance, most optimizers would eliminate a `memset` on a region not accessed later, illustrated in this example [@yang2017]:

```
static void invert_key(
  u16 *ek,
  u16 dk[IDEA_KEYLEN]) {
  // Allocated on stack to hold inverted key
  u16 temp[IDEA_KEYLEN];
    ...
  memcpy(dk, temp, sizeof(temp));
  // Clear the value from memory
  memset(temp, 0, sizeof(temp));
}
```

While functionally redundant, this `memset` is necessary from a security perspective, to explicitly erase the secret value from memory and prevent memory-dump attacks.

This project intends to explore this issue commonly known as the Correctness-Security Gap *[@xu2023]* by formally verifying that specific optimization passes on a toy language maintain designated security properties.

More specifically, we will implement a Skip removal and Dead Store Elimination (DSE) on a toy language. We want to prove that DSE is indeed insecure, with a program similar to the above example. We will patch DSE and prove that both optimizations are sound and secure.

Approach

Define a Language

We defined a simple language, based on the one defined in the CS-428 labs[1]:

```
Inductive cmd :=
| Skip : cmd
| Assign : var → expr → cmd
| Cmov : var → expr → expr → expr → cmd
| Sequence : cmd → cmd → cmd
| If : expr → cmd → cmd → cmd
| While : expr → cmd → cmd.
```

We added `Cmov`, a ternary operator, which makes the language more complete, allowing secure conditionals on secrets (disallowed in secure languages, as explained in the next sections). The language includes expressions with binary operators and variables. This project report assumes the reader broadly knows this language.

[1] We initially included function calls (`AssignCall`), but removed them to reduce unnecessary complexity, as the optimization of the function bodies was left for future work.

Interpreting this language is done using the `eval` inductive relation[2].

Introduce a Security Context

To define security properties, we must extend the language semantics, as it currently lacks security notions.

We introduce `trace_events`, to represent branching decisions at `If` and `While` statements. We define the `eval_trace` inductive, which mirrors `eval`, while also constructing a list of `trace_event` in an evaluation context[3]. This trace will enable the later analysis of timing side-channel vulnerabilities.

Next, we introduce `Secret` and `Public` security levels for variables, following CT-wasm, a type-driven extension of WebAssembly designed to enforce constant-time programming disciplines [@watt2019]. A third inductive relation, `eval_sec`, extends `eval_trace` by tracking the set of secret variables in addition to the valuation and trace.[4]

Secrecy propagates via assignment. The assignee inherits the expression's security level. Constants are public, while binary operations (`Binop`), yields secret if any of the operands is secret.

We proved the equivalence of these three evaluation inductive relations by projecting or lifting the trace and/or the security environment. Isolating these components into three different evaluators establishes separation of concern, resulting in cleaner proofs.

The example below evaluates a simple while loop, which should branch twice: one iteration of the while loop, then exit. Evaluating with `eval_trace` gives an output trace `[true; false]`:

```
Definition simple_while :=
  while x loop x <- 0 done.

Example simple_while_true_false_branch :
  ∃ v, eval_trace
    (* starting context:
       [x := 1], no trace *)
    ($0 $+ ("x", 1), nil)
    simple_while
    (* end context: valuation irrelevant,
      trace has two elements*)
    (v, Branch true :: Branch false :: nil).
Proof. crush_eval. Qed.
```

[2] `eval` was also closely inspired by the CS-428 course.

[3] `context := valuation * list trace_event`

[4] `sec_context := valuation * sec_env * list trace_event`

Formalize Security Properties

Constant-time and non-interference

Our goal is to secure the language against timing side-channel attacks. Since an attacker could use timing techniques to infer which branches were taken, we treat the execution trace as public knowledge. Hence, we require both the final trace and output valuation not to depend on secret variables. This property is known as non-interference [@barthe2019].

```
Definition non_interference
  se c se_out : Prop :=
 ∀ v1 v2 t_in final_v1 final_v2
   trace1 trace2,
 (* if valuations agree on public vars *)
  agree_on_public se v1 v2 →
 (* and we evaluate `c` with each valuation*)
  eval_trace
    (v1, t_in) c (final_v1, trace1) →
  eval_trace
    (v2, t_in) c (final_v2, trace2)
  →
 (* then the final trace agrees*)
  trace1 = trace2 ∧
 (* public vars after execution agree*)
  agree_on_public
    se_out final_v1 final_v2.
```

One approach to ensure programs satisfy non-interference is to disallow any branching on secret variables. To achieve this, we defined a static tree checker `ct_cmd`, inspired by CT-wasm [@watt2019], and proved its soundness:

```
Theorem ct_cmd_soundness : ∀ se c se_out,
  ct_cmd se c se_out →
  non_interference se c se_out.
```

Here is a sample program that fails this check. The two branches have different lengths, exposing a timing side-channel attack.

```
Definition secret_guard_program :=
  when secret then x <- 1;; y <- 1
  else x <- 0 done.
```

Secret Preservation

The motivational example from the introduction highlights a problem not yet addressed. An assignment could be removed if it appears redundant (dead code). However,

if this assignment intentionally overwrites a secret to clear it from memory, its removal breaks security. Hence, we require that optimizations preserve the secrecy levels and the values of secret variables:

```
Definition opt_keeps_secrets_secret
  opt : Prop :=
 ∀ sctx sctx_1 sctx_2 pgm opt_pgm,
  opt pgm = opt_pgm →
 (* After executing source and optimized
    program *)
  eval_sec sctx pgm sctx_1 →
  eval_sec sctx opt_pgm sctx_2 →
 (* the result agree on secrecy levels *)
  same_secrets sctx_1 sctx_2 ∧
 (* and on the value of secrets *)
  agree_on_secret sctx_1 sctx_2.
```

This property will later enable proving that a standard implementation of Dead Store Elimination can make a program insecure.

Secure Programs and Optimizations

Having established these security properties, we can now formalize the definition of a secure program and a secure optimization.

A secure program is considered secure if it satisfies non-interference:

```
Definition is_secure_pgm
  (pgm: program) (secrets: sec_env)
  : Prop :=
  ∃ secrets',
    non_interference
      secrets (cmd_of pgm) secrets'.
```

A secure optimization must satisfy two criteria

1. It transforms a secure program into another secure program
2. It does not alter the final values or security levels of secrets with respect to the source program.

```
Definition is_secure_opt
  (opt: program → sec_env → program)
  : Prop :=
  (∀ (p: program) (secrets: sec_env),
    is_secure_pgm p secrets →
    is_secure_pgm (opt p secrets) secrets)
    ∧
    opt_keeps_secrets_secret opt.
```

Implement the Optimizations

Unskip

This cleanup optimization removes all redundant `Skip` commands. In particular, it simplifies sequences and conditionals where both branches are `Skip`.

```
Fixpoint opt_unskip_cmd (c: cmd) :  cmd :=
  match c with
  | Sequence c1 c2 ⇒
    match (opt_unskip_cmd c1),
          (opt_unskip_cmd c2) with
    | Skip, c2' ⇒ c2'
    | c1', Skip ⇒ c1'
    | c1', c2' ⇒ c1' ;; c2'
    end
  | If cond thenC elseC ⇒
    match opt_unskip_cmd thenC,
          opt_unskip_cmd elseC with
    | Skip, Skip ⇒ Skip
    | thenC', elseC' ⇒
        If cond thenC' elseC'
    end
(* Don't simplify empty loop:
   it might be non-terminating *)
  | While cond body ⇒
      While cond (opt_unskip_cmd body)
  | _ ⇒ c
  end.
```

Unskip alters the program's trace, making it an interesting optimization, but also makes security proofs considerably more difficult:

```
(when "x" then Skip else Skip done).
```

is optimized to `Skip`. The source program will always produce a nonempty trace, while the optimized one will always produce an empty trace. This implies that we cannot rely on simple trace equality via forward simulation to prove non-interference preservation.[5] Instead, we needed complex backward simulation to map the optimized program's behavior back to the source.

Dead Store Elimination

Dead Store Elimination (DSE) removes assignments that do not contribute to the program's final output. Identifying which assignments are "dead" requires first defining

[5]Our initial version of `Unskip` did not reduce `If` statements, the first security proof could use this trace equality and was simpler.

an output set for each program, as otherwise every assignment would be considered dead.

```
Definition program := cmd * var_set.
```

The main challenge for this optimization is to determine which assignment can be safely omitted. We implement this using a backward liveness analysis, which begins at the end of the program with a given output set and propagates the set of "live" variables (the ones required for future computations) backward.

For an assignment `x ← e`, we update the live variable set as follows:

- If `x` is live, the variables in expression `e` become live, and `x` is removed from the set (as it is overwritten).
- If `x` is not live, the assignment is dead and can be safely removed, leaving the set of live variables unchanged.

```
Fixpoint liveness
  (c: cmd) (out: var_set) : var_set :=
  match c with
  | Skip ⇒ out
  | Assign x e ⇒
      if VarSet.mem x out
      then VarSet.union
        (VarSet.remove x out)
        (vars_in_expr e)
      else out
  ...
  | Sequence c1 c2 ⇒
      liveness c1 (liveness c2 out)
  ...
end.
```

Liveness analysis of a while loop is especially tricky. Liveness can propagate during many iterations, so we needed a fixed point algorithm to ensure all variables are reached. We use fuel[6] to indicate to Rocq that the fixpoint will eventually end.

The `DSE` pass then leverages this analysis, performing a simple membership lookup to infer whether the assigned variable is live or not:

```
Fixpoint dse_helper
  (c: cmd) (out: var_set) : cmd :=
  match c with
  | Skip ⇒ Skip
  | Assign x e ⇒
      match VarSet.mem x out with
      | false ⇒ Skip
      | true ⇒ Assign x e
      end
  ...
  | Sequence c1 c2 ⇒
      Sequence
        (dse_helper c1 (liveness c2 out))
        (dse_helper c2 out)
  ...
  end.
```

While this approach is functionally correct, it suffers from the limitation highlighted by the motivational example: it may remove assignments that serve a security purpose. We address this issue in the next section.

Secure Dead Store Elimination:

A naive approach to securing DSE would be to prevent the removal of any assignment involving secret variables. This would only change the variable filtering in the `Assign` case:

```
match (VarSet.mem x out),
      (sec_lookup secrets x) with ...
```

However, because secrets propagate dynamically through assignments, we must also track which variables become secret during the program's execution. To this end, we introduce a forward-working secret tracking tool `sec_post`, which (conservatively) approximates the set of secret variables after a `cmd`.

When removing an assignment, we need to make sure that (1) the assignee is not already a secret and (2) the assigned expression is not a secret, which would taint the assignee.

This technique combines forward (secrets) and backward (liveness) analysis to create a secure version of Dead Store Elimination:

```
Fixpoint secure_dse_helper
  (c: cmd) (out: var_set)
  (secrets: sec_env) : cmd :=
  match c with
  | Skip ⇒ Skip
  | Assign x e ⇒
      match (VarSet.mem x out),
            (sec_lookup secrets x),
            (interp_sec e secrets) with
      | false, Public, Public ⇒ Skip
      | _, _, _ ⇒ Assign x e
      end
  ...
  | Sequence c1 c2 ⇒
      let secrets_mid :=
```

[6] fuel = # variables in the `While` statement.

```
      sec_post c1 secrets in
    Sequence
      (secure_dse_helper c1
        (liveness c2 out) secrets)
      (secure_dse_helper c2
        out secrets_mid)
 ...
end.
```

Results

Composing optimizations

We proved that any composition of two secure optimizations is secure, provided the additional constraint that the first optimization produces a secure environment as a starting point for the second one. Note that this is not asking for the first optimization to be sound, as we keep soundness and security disjoint properties in this project.

```
Theorem is_secure_opt_compose :
  ∀ opt1 opt2,
  is_secure_opt opt1 →
  is_secure_opt opt2 →
  opt_eval_sec_sound opt1 →
  is_secure_opt (compose_opt opt2 opt1).
```

This result allows us to chain any number of secure optimizations, making the proof of the compiler's security scalable.

Unskip

We have formalized both the soundness and security of the `Unskip` optimization:

```
Theorem opt_unskip_sound: ∀ c v v',
  eval v c v' →
  eval v (opt_unskip_cmd c) v'.

Theorem opt_unskip_secure :
  is_secure_opt opt_unskip.
```

The soundness notion proved is even stronger than required by our framework, as the optimized program agrees with the original one on the entire evaluation context rather than solely on the output set.

Dead Store Elimination

Soundness

Soundness has been formalized for both versions of this optimization, meaning the optimized program's final valuation agrees on the output set with the source one.

```
Theorem opt_secure_DSE_sound :
  ∀ pgm secrets v v',
  eval v (cmd_of pgm) v' →
  ∃ v_opt,
    eval v
      (cmd_of (opt_secure_DSE pgm secrets))
      v_opt ∧
    agree_on (output_of pgm) v' v_opt.
```

Insecure

The first approach has been proven to be insecure by a counterexample: `secret ← 0`.

It is the same issue as in the introductory example. The secret overwrite being dead, it is removed. Running the optimized program does not output the same secrets as the source program, which breaks our second security property.

```
Theorem insecureDSE_insecure :
  ~ is_secure_opt (fun p _ ⇒ opt_dse p).
```

Secure

The second approach has been proven secure:

```
Theorem opt_secure_DSE_secure :
  is_secure_opt opt_secure_DSE.
```

The above proof relies on decomposing DSE into two stages. First, we define a core DSE pass that replaces redundant assignments with `Skip`. We then reuse the composition theorem from our theorem to compose this core pass with an `unskip` cleanup pass.

The core of the proof lies in showing that this intermediate DSE pass is secure. We show this by proving its completeness: any execution of the optimized program can be mapped to an equivalent execution of the original program that satisfies the following:

1. It preserves the same security label
2. It produces the same trace
3. It maintains agreement on the output variables.

Limitations

Our security-aware Dead Store Elimination relied on conservative estimations, particularly in the case of `While` loops. This makes more variables secret than strictly necessary, limiting the scope of the optimization.

Furthermore, our memory model (valuations) is significantly simpler than the memory representation required to capture real-world side-channel vulnerabilities such as the one in the introductory example.

Timeline

Planned vs. actual timeline

- W08: planned: Define language. Actual: Base language, semantics, and execution traces.
- W09/10: planned: Formalize optimizations & security. Actual: Security context, Constant-Time checker, DSE implementation.
- W11/12: planned: Prove correctness & security. Actual: Soundness proofs, Secure DSE, CMOV added, and codebase refactored.
- W13/14: planned: Finalize, report, presentation. Actual: Security proofs completed, DSE refined, repo cleaned, and final deliverables.

Main divergences from proposal: We added the Cmov to our language to make it more complete, while still respecting constant-time constraints. We could not achieve any stretch goals mentioned in the proposal.

We also planned to add many *standard* optimizations and prove them secure. This took more time than expected, so we only implemented two optimizations.

Workload repartition Everyone worked on and reviewed all features; nevertheless, each member was mainly responsible for one feature:

- Overall language infrastructure: Guillaume Carraux.
- Dead Store Elimination proofs: Sean Perazzolo.
- Unskip proofs: Maria Bennani.

Related work

The Jasmin Project explores the same idea at a broader scale: Jasmin is a programming language for high-assurance, high-speed cryptography, with a compiler formally verified in Rocq/Coq [@almeida2017].

Future work

Several directions could extend this project beyond simply enlarging the language, adding more optimizations, or strengthening the existing one.

Below are several directions that we find interesting to pursue:

- Model memory and extend the trace to record memory access, covering more side-channel attacks.
- Add declassification to allow some secret information to be released intentionally.

Conclusion

This project showed how to verify optimizations against both semantic and security properties. We defined a small language with traces and secrecy information, proved that standard dead-store elimination can be insecure, and developed secure optimizations with machine-checked proofs.

AI use disclosure

LLMs assisted in Rocq documentation, large proof planning, and helped in writing difficult proofs in this project.

References